

Assignment 1

Simulating Memory: Devices to Systems

ngspice · CACTI · NVSim · Ramulator · gem5

Priyesh Shukla · IIIT Hyderabad · Monsoon 2026

Overview

This assignment carries one design question through five simulation tools, each operating at a different level of abstraction. The central question is:

"Should the 2 MB L2 cache in our accelerator be SRAM, or STT-MRAM?"

Each part answers one question that none of the other four tools can. The output of each part is a number you carry into the next part's input file. Report ratios between configurations, not absolute numbers — ratios survive the compounding errors.

Submission: One PDF, 6-8 pages — five tables, four plots, and the two-paragraph argument from Part E Task 4. *Working in pairs is encouraged.*

Marking: 20 % Part E argument · 60 % remaining questions · 20 % report & reproducible codes (provide GitHub repo link in report)

Part A — ngspice: Read a 6T SRAM cell and measure the margin

Central question — Does the bitcell actually read correctly, at 0.7 V and 85 °C?

Use the netlist below (45 nm PTM BSIM4, Q = 0, QB = 1 stored). The cell ratio is deliberately marginal — Task 2 is meant to fail.

```
* AMCAS TakeHomeLab - 6T read, bitline discharge and margin
.include 45nm_bulk.txt          $ PTM BSIM4 card
.param VDD=1.1 VBL=1.1
Vdd vdd 0 'VDD'
Vw1 w1 0 PWL(0 0 1n 0 1.05n 'VDD')
MP1 qb q vdd vdd pmos W=0.15u L=0.045u
MN1 qb q 0 0 nmos W=0.20u L=0.045u
MP2 q qb vdd vdd pmos W=0.15u L=0.045u
MN2 q qb 0 0 nmos W=0.20u L=0.045u
MA1 bl w1 q 0 nmos W=0.16u L=0.045u
MA2 blb w1 qb 0 nmos W=0.16u L=0.045u
Cbl bl 0 180f IC='VBL'
Cblb blb 0 180f IC='VBL'
.ic v(q)=0 v(qb)='VDD'
.control
  tran 5p 4n uic
  meas tran dv FIND v(blb)-v(bl) AT=2.0n
  meas tran qmax MAX v(q) FROM=1n TO=4n
  wrdata read.csv v(bl) v(blb) v(q) v(qb)
.endc
.end
```

Task 1. Report $\Delta V(\text{BL}, \text{BLB})$ at $t = 2 \text{ ns}$. Compare it with the 69 mV reference value given in Lecture 1.

Task 2. Plot $v(q)$. Does it move during the read? Grow MA1 and MA2 to $W = 0.24 \mu\text{m}$ and re-run — what happens now? *[deliberately marginal; expect a failure]*

Task 3. Sweep V_{DD} from 1.1 V down to 0.6 V in 50 mV steps using a .control loop. Plot ΔV vs V_{DD} and find the voltage at which ΔV falls below the 25 mV sense-amp offset from Lecture 5.

Task 4. Repeat Task 1 at .temp 85. In one paragraph state which of ΔV and the sense-amp offset moved more, and which one sets the failure.

Note: ngspice is the only tool in this assignment that solves physics. Everything above it solves arithmetic. Your CACTI numbers in Part B are curve-fits to results like these.

Part B — CACTI: Size the 2 MB SRAM L2

Central question — How fast, how big, and how leaky is a 2 MB SRAM L2 at 45 nm?

CACTI does not simulate a single transistor. It performs an exhaustive search over array organisations ($N_{dwl} \times N_{dbl} \times N_{spd}$ for data; $N_{twl} \times N_{tbl} \times N_{tspd}$ for tags) and scores each candidate with analytical delay, energy, and area models, then returns the organisation that minimises your objective.

```
# cache.cfg (CACTI 7)
-size (bytes) 2097152
-block size (bytes) 64
-associativity 8
-read-write port 1
-UCA bank count 4
-technology (u) 0.045
-operating temperature (K) 350
-cache type "cache"
-Data array cell type - "itrs-hp"
-Data array peripheral type - "itrs-hp"
-Optimize ED or ED^2 (ED, ED^2, NONE): "ED^2"
-design objective (weight delay, dynamic power,
leakage power, cycle time, area) 0:0:0:100:0

$ ./cacti -infile cache.cfg
```

Task 1. Run CACTI. Record access time (ns), dynamic read energy (pJ), leakage (mW), and area (mm²). This is your SRAM baseline for Parts C, D, and E.

Task 2. Sweep capacity from 256 kB to 16 MB (everything else fixed). Plot access time against $\log_2(\text{capacity})$. Is Amrutur & Horowitz's "one gate delay per doubling" visible?

Task 3. Change only the objective — run once for pure delay, once for pure area, once for ED²P. Report the three winning N_{dwl}/N_{dbl} triples and explain why the optimal organisation changes.

Task 4. Take CACTI's bitline delay for your Task 1 winner and hand-check it against $0.38 \cdot R \cdot C \cdot L^2$ from Lecture 4. Report the discrepancy and name one effect CACTI models that your hand calculation does not.

Note: Task 3 is the one that matters. There is no such thing as 'the optimal cache' — only the optimum of the objective you wrote down.

Part C — NVSim: The same 2 MB as STT-MRAM

Central question — Same capacity, same periphery — different bitcell. What changes?

NVSim is CACTI with the bitcell model extracted into a separate text file you supply. The array organisation, peripheral search, and output format are identical. Your results are exactly as good as the .cell file you feed it.

```
# sample.cell (STT-MRAM, 16 lines)
-MemCellType: MRAM
-CellArea (F^2): 54
-CellAspectRatio: 2.0
-ResistanceOn (ohm): 3000
-ResistanceOff (ohm): 6000
-ReadMode: current
-ReadCurrent (uA): 40
-ReadPower (uW): 145
-ResetMode: current
-ResetCurrent (uA): 200
-ResetPulse (ns): 10
-SetMode: current
```

```
-SetCurrent (uA): 200
-SetPulse (ns): 10
-AccessType: CMOS
-AccessCMOSWidth (F): 6

$ ./nvsim ./config/STT_cache.cfg
```

Task 1. Run NVSim at 2 MB with the shipped STT-MRAM cell. Tabulate read latency, write latency, read energy, write energy, leakage, and area side-by-side with your CACTI SRAM baseline from Part B.

Task 2. Two of those six metrics will be dramatically worse and two dramatically better. Identify which ones and explain the difference in device terms — not 'because the tool said so'.

Task 3. The TMR ratio here is 2:1 (3 kΩ / 6 kΩ). Re-run with R_{on} = 4 kΩ and R_{off} = 12 kΩ. Which output metric moved most? Explain what a device paper's TMR headline actually buys at the array level.

Task 4. Halve ResetCurrent to 100 μA and re-run. Note what happened to area and explain the mechanism — this is the single most important coupling in NVM array design.

Note: Task 4: write current → sizes the access transistor → sets cell area → sets the density advantage you bought the technology for. Follow that chain explicitly in your write-up.

Part D — Ramulator 2.0: L2 miss stream on a DDR4 channel

Central question — Does the changed miss traffic still fit in one DDR4 channel?

A DRAM simulator contains no charge, no capacitor, and no sense amplifier. It is a table of timing constraints (t_{RC}D, t_{RP}, t_{RA}S, t_{CC}D), a state machine per bank, and a scheduler that issues the next legal command. Generate the trace from gem5's CommMonitor on your L2's memory-side port.

```
# ddr4.yaml (Ramulator 2.0)
Frontend:
  impl: SimpleO3
  traces: [ l2miss.trace ]
  clock_ratio: 8
MemorySystem:
  impl: GenericDRAM
  clock_ratio: 3
  DRAM:
    impl: DDR4
    org: { preset: DDR4_8Gb_x8, channel: 1, rank: 2 }
    timing: { preset: DDR4_3200AA }
  Controller:
    impl: Generic
    Scheduler: { impl: FRFCFS }
    RefreshManager: { impl: AllBank }
    plugins: [ { ControllerPlugin: { impl: TraceRecorder } } ]

# l2miss.trace (one access per line)
0x7f2a4c00 R
0x7f2a4c40 R
0x7f2a5000 W

$ ./ramulator2 -f ddr4.yaml
```

Task 1. Run Ramulator. Report memory_{cycles}, total read/write latency, and the row-buffer hit rate from the stats dump.

Task 2. Switch Scheduler from FRFCFS to FCFS. Row-buffer hit rate will collapse. Quantify the latency cost in cycles and explain it in terms of t_{RC}D and t_{RP}.

Task 3. Change the address mapping so the row bits sit below the bank bits. Re-run. Quantify the loss of bank-level parallelism.

Task 4. Double the channel count. If latency barely improves, identify which of the four DRAM timing parameters is binding and explain how you would determine this from the output.

Note: Generate `l2miss.trace` yourself: the `CommMonitor` on `gem5`'s L2 memory-side port produces exactly this format.

Part E — gem5: Run the whole thing

Central question — Does the program you actually care about run faster?

`gem5` is the only tool in this assignment that can answer the central question in cycles. The L2 hit latency you pass to `gem5` comes from `CACTI` and `NVSim`; the capacity trade comes from their area outputs. `gem5` invents neither. Warm up atomically for tens of millions of accesses before switching to `O3CPU`.

```
# Baseline — CACTI SRAM L2
$ build/X86/gem5.opt configs/deprecated/example/se.py \
  --cpu-type=O3CPU --caches --l2cache \
  --l1d_size=32kB --l1i_size=32kB \
  --l2_size=2MB --l2_assoc=8 \
  --l2-hit-latency=7 \          # <- from CACTI, in cycles
  --mem-type=DDR4_2400_8x8 --mem-size=4GB \
  --cmd=./bfs --options="-g 18"

# Alternative — NVSim STT-MRAM L2 (8 MB in same area, 2x slower hits)
$ ... --l2_size=8MB --l2-hit-latency=14

# Read results
$ grep -E "simSeconds|ipc|l2.*miss_rate" m5out/stats.txt
```

Task 1. Run both configurations on two GAPBS kernels (e.g. `bfs` and `sssp`). Report IPC, L2 miss rate, and `simSeconds` for each.

Task 2. The MRAM configuration is 4× larger and ~2× slower per hit. For which kernel does that trade win, and what property of the kernel determines the outcome?

Task 3. Re-run the winning configuration with `--cpu-type=TimingSimple`. The advantage will change in magnitude. Explain what out-of-order execution was doing to your Part E Task 1 result.

Task 4. Write two paragraphs — this is the primary deliverable. (i) Answer the central question from the assignment overview, with the evidence from Parts A–E. (ii) State the three assumptions in your simulation chain you would attack first if you were the reviewer. [30 % of total marks]

Note: 'STT-MRAM is M× faster' is not a result. 'STT-MRAM is M× faster on bfs, given these device numbers, this cell file, this scheduler, and an O3 core' is a result.

Setup & References

Install: `ngspice` (apt/brew), `CACTI 7` and `NVSim` (make), `Ramulator 2.0` (CMake, C++20), `gem5` (scons — allow 40 min). Docker images for all five are on the course page.

Model cards: PTM 45 nm bulk BSIM4 from `ptm.asu.edu` for Part A. All other parts use the shipped 45 nm configs so your five results remain comparable.

Tool	Repository / Site	Primary paper to cite
<code>ngspice</code>	<code>ngspice.sourceforge.io</code>	PTM BSIM4 — <code>ptm.asu.edu</code>
<code>CACTI 7</code>	<code>github.com/HewlettPackard/cacti</code>	Balasubramonian et al., ACM TACO, vol. 14, no. 2, 2017
<code>NVSim</code>	<code>github.com/SEAL-UCSB/NVSim</code>	Dong, Xu, Xie & Jouppi, IEEE TCAD, vol. 31, no. 7, 2012
<code>Ramulator 2.0</code>	<code>github.com/CMU-SAFARI/ramulator2</code>	Luo et al., IEEE CAL, vol. 23, no. 1, 2024
<code>DRAMsim3</code>	<code>github.com/um-d-memsys/DRAMsim3</code>	Li et al., IEEE CAL, vol. 19, no. 2, 2020
<code>gem5</code>	<code>gem5.org · learning.gem5.org</code>	Lowe-Power et al., arXiv:2007.03152, 2020

Accuracy reminders: `ngspice` ±5 % (model-card limited) · `CACTI` ±10 % · `Ramulator` ±10 % · `gem5` ±20 %. Errors compound multiplicatively — report ratios, not absolutes.